https://github.com/IT24103448/IT3012---Practical-Base/tree/lab-07

Practical 07

1. Reachability vs. Feasibility

Reachability checks physical constraints: the next coordinate must be inside the grid and must not be a wall. Feasibility checks whether the game's rules permit entering that reachable tile. For each valid neighbor, A* clears the KB's facts, loads that tile's facts, and runs forward chaining. If Retreat is deduced, A* skips that tile before adding it to the priority queue.

For (1, 0), TargetVisible and HasDust imply SafeToEngage. SafeToEngage together with BloodseekerMissing then implies Retreat, so the tile is excluded even though there is no physical wall. For (0, 1), only SafeToEngage is deduced, so movement is allowed.

The worksheet uses a specific policy: reject a tile when Retreat is provable.It does not prove that every tile without Retreat is safe in the real world. Missing information is not the same as proof that a threat is absent.

2. Declarative vs. Procedural Paradigms

The KB describes what relationships are true using facts and rules, while forward_chain() provides the reusable procedure for applying those rules. The agent's movement code only asks whether Retreat was deduced. This separates domain knowledge from pathfinding and supports chains of deductions.

With 50 additional rules, we can register those rules using tell_rule() without adding 50 branches to the A* movement loop. A direct if-else implementation would mix navigation with rule ordering and dependencies, making it harder to maintain and test. More rules still increase the inference work, but the same inference algorithm can process them.

3. Modus Ponens and Horn Clauses

A definite Horn clause has a conjunction of premises implying one positive conclusion, for example: TargetVisible AND HasDust -> SafeToEngage.

Modus Ponens says that if the premises are known to be true and the implication is a rule, its conclusion can be inferred. The Python expression all(premise in self.facts for premise in premises) checks that every premise is known. If so, the engine adds the conclusion to its facts.

The while loop repeats because one conclusion can become a premise for another rule. For example, SafeToEngage enables the rule that derives Retreat when BloodseekerMissing is also present. The loop stops at a fixed point when a full pass adds no facts. Sets prevent duplicate deductions, so cyclic rules also terminate for a finite knowledge base.